

PRODUCT REQUIREMENTS DOCUMENT

Ai-VendorHub

AI-Powered Multi-Vendor Marketplace Platform

Version 1.0
Date 2026-07-06
Owner Shiva Choudhry
Repo [Ai-VendorHub-master](#)
Status **Active Development**

Contents

- 1 Product Overview
- 2 Objectives and Goals
- 3 Target Users
- 4 Actual Problem Statement
- 5 Proposed Solution
- 6 Features
- 7 User Flow
- 8 Data Flow Diagrams
- 9 Functional Requirements
- 10 Non-Functional Requirements
- 11 Tech Stack
- 12 AI Intelligence Layer
- 13 Risk Assessment

1. Product Overview

One-liner: Ai-VendorHub is an AI-powered, microservices-based e-commerce marketplace where customers can discover products, sellers can manage inventory and analytics, and admins can operate marketplace content, notifications, and platform workflows.

Field	Detail
Product Name	Ai-VendorHub
Repository	Ai-VendorHub-master
Owner / Team	Shiva Choudhry
Date	2026-07-06
Version	1.0
Primary Stack	Node.js, Express.js, React, MongoDB, RabbitMQ, Redis, Socket.IO
AI Stack	Google Gemini / LangChain, semantic ranking, persistent AI memory
Status	Active Development

Description

Ai-VendorHub is a full-stack marketplace platform built around independent backend services for authentication, products, cart, orders, payments, notifications, seller dashboard, and AI. It combines standard e-commerce flows with real-time operational events, seller analytics, and an AI shopping assistant that can search, recommend, compare, explain, and act on products.

2. Objectives and Goals

- 1 **Enable Marketplace Commerce** - Support customer discovery, cart, checkout, payments, order tracking, and notification workflows.
- 2 **Empower Sellers** - Provide product management, order visibility, inventory analytics, low-stock alerts, and product health insights.
- 3 **Add AI-Assisted Shopping** - Help users search naturally, compare options, build bundles, understand product fit, and take cart or wishlist actions.
- 4 **Use Scalable Service Boundaries** - Keep auth, product, cart, order, payment, notification, seller dashboard, and AI concerns isolated.
- 5 **Support Operational Visibility** - Use events, feeds, metrics, and notifications to make marketplace activity observable.
- 6 **Remain Demo-Friendly** - Keep services understandable, independently runnable, and suitable for interview or academic explanation.

3. Target Users

User Persona	Type	Description
Customer / Buyer	Primary	Browses products, uses AI recommendations, manages cart, places orders, pays, tracks status, and receives notifications.
Seller	Primary	Creates and updates products, manages stock, views seller orders, tracks analytics, and handles low-stock alerts.
Admin	Primary	Manages homepage sections, platform notifications, order operations, and broader product oversight.
Marketplace Operator	Secondary	Monitors service health, AI metrics, feature flags, and event-driven workflows.
Interview / Demo Evaluator	Secondary	Reviews architecture, service boundaries, data flow, and real-world engineering choices.

4. Actual Problem Statement

Core Problem: Most student marketplace projects stop at CRUD. They do not show realistic service separation, cart/order/payment consistency, seller operations, event-driven updates, or useful AI beyond a weak chatbot.

Existing simple e-commerce projects usually miss these production-style concerns:

- **No real service boundaries** - auth, catalog, cart, orders, and payments often live in one monolith.
- **Weak seller workflow** - sellers can list products, but do not get analytics, inventory risk, or live order context.
- **Limited AI usefulness** - chatbots often answer generic questions without product grounding, personalization, or actions.
- **Low operational visibility** - no notifications, event feeds, metrics, or workflow status clarity.

5. Proposed Solution

Ai-VendorHub addresses these gaps using a layered marketplace architecture:

- **Layer 1 - Identity and Security:** JWT-based authentication, role-based access, profile management, address book, Google OAuth support, password reset, and email verification.
- **Layer 2 - Marketplace Core:** product catalog, wishlist, recently viewed products, cart validation, order lifecycle, payment creation/verification, and notifications.
- **Layer 3 - Seller and Admin Operations:** seller metrics, conversion funnel, inventory movement, inventory risk, forecasting, low-stock alerts, homepage content management, and notification controls.

- **Layer 4 - AI Intelligence:** natural-language search, semantic ranking, persistent memory, personalized recommendations, product comparison, smart budget bundles, mood shopping, seller listing generation, and cart/wishlist actions.

Why this works: The project demonstrates both product thinking and backend engineering: user-facing commerce, seller operations, AI-assisted discovery, and microservice communication all exist as separate explainable modules.

6. Features

6.1 F1 - Secure Authentication

- User, seller, and admin login flows
- JWT authorization middleware
- Profile and address management
- Email verification, password reset, refresh token, logout, and logout-all flows
- Google OAuth callback support when configured

6.2 F2 - Product Discovery and Catalog

- Product CRUD for seller/admin roles
- Image upload support
- Pagination, filtering, text search, category, brand, price, and rating filters
- Trending products, product comparison, related products, variants, recently viewed history, and wishlist

6.3 F3 - Cart, Order, and Payment Workflow

- Stock-aware add/update/remove cart operations
- Cart validation, cart status, save-for-later, and abandoned-cart scan
- Order creation from cart, order history, cancellation, address updates, expiry scan, and status updates
- Razorpay payment creation/verification and local test-success payment route

6.4 F4 - Notifications and Events

- User/seller/admin notification inbox
- Unread counts, read-all, single-read, delete, admin create, email retry
- RabbitMQ-style event modules for product, cart, order, payment, notification, and dashboard workflows

6.5 F5 - Seller Dashboard

- Seller metrics
- Seller order and product views
- Live order feed
- Conversion funnel
- Product health
- Inventory movement
- Top-losing products
- Inventory risk and forecast
- Low-stock alert read/resolve actions

6.6 F6 - AI Shopping and Seller Assistant

- Conversational shopping assistant
- Search intent parser
- Product page AI insights
- Similar products
- Product comparison
- Smart budget bundles
- Mood shopping
- Review summaries
- Description/category/tag generation
- Persistent memory, semantic ranking, personalization, and cart/wishlist actions

7. User Flow

7.1 Primary Flow - Customer Purchase Journey

- 1 Customer registers or logs in and receives a JWT.
- 2 Customer browses products using filters, search, wishlist, recently viewed, or AI chat.
- 3 AI interprets intent, semantically ranks products, and shows personalized product cards.
- 4 Customer adds items to cart manually or asks AI to add the selected product.
- 5 Cart service validates stock and recalculates totals.
- 6 Customer creates an order from the cart with shipping address.
- 7 Order service publishes lifecycle events and clears the cart.
- 8 Payment service creates/verifies Razorpay payment or test success payment.

- 9 Notification and seller dashboard services surface order/payment/inventory events.

7.2 Secondary Flow - Seller Product and Inventory Journey

- 1 Seller registers/logs in with seller role.
- 2 Seller creates products with title, description, category, price, stock, tags, variants, and images.
- 3 AI helps generate descriptions, tags, categories, and seller-facing product improvements.
- 4 Seller dashboard tracks product health, orders, live feed, stock risk, forecasts, and low-stock alerts.
- 5 Seller updates inventory or resolves low-stock alerts based on dashboard signals.

8. Data Flow Diagrams

The following diagrams summarize the two most important flows: checkout/service orchestration and AI personalization/action execution.

Diagram 1 - Customer Checkout and Event Data Flow

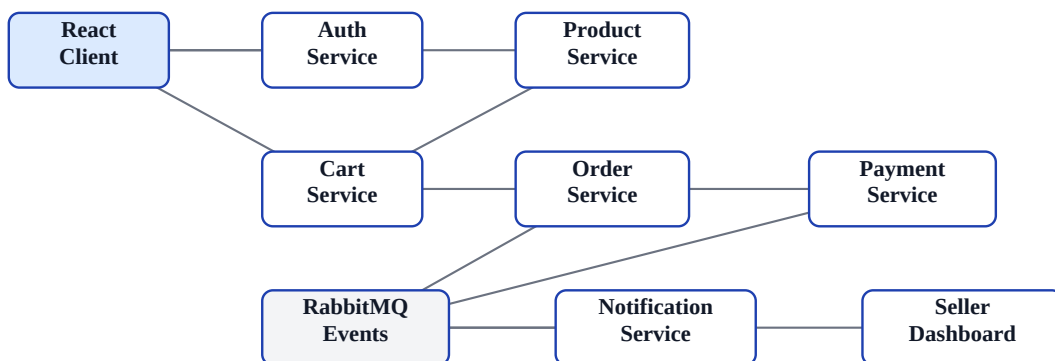
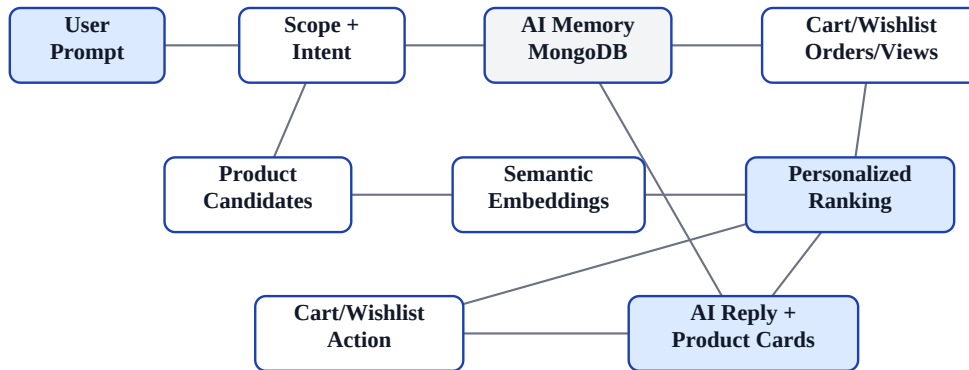


Diagram 2 - AI Semantic Search, Memory, and Action Data Flow



9. Functional Requirements

ID	Feature	Requirement	Priority
FR-01	Authentication	System shall support user, seller, and admin registration/login with JWT authorization.	High
FR-02	Profile	System shall allow authenticated users to manage profile and addresses.	High
FR-03	Catalog	System shall allow sellers/admins to create, update, archive/delete, and list products.	High
FR-04	Discovery	System shall support search, filters, trending, related products, wishlist, and recently viewed products.	High
FR-05	Cart	System shall validate stock and pricing when cart items are added or updated.	High
FR-06	Order	System shall create orders from cart items and support order status, cancellation, and address update flows.	High
FR-07	Payment	System shall create/verify payments and expose local test success flow.	High
FR-08	Notifications	System shall provide notification inbox, unread counts, read/delete actions, admin create, and retry email delivery.	Medium
FR-09	Seller Dashboard	System shall show seller metrics, orders, products, inventory analytics, feed, and low-stock alerts.	High
FR-10	AI Search	AI shall parse natural language queries and return real ranked products from the catalog.	High
FR-11	AI Actions	AI chat shall support add-to-cart, wishlist, save-for-later, and remove-from-cart intents.	Medium
FR-12	AI Memory	AI shall persist conversation and user preference memory when MongoDB is configured.	Medium

10. Non-Functional Requirements

10.1 Performance

- Product and cart APIs should return within a few hundred milliseconds in local demo conditions.
- AI endpoints should degrade gracefully with fallback ranking when LLM calls fail.
- Cart/order/payment flows should avoid blocking unrelated services where possible.

10.2 Security

- JWT must protect private customer, seller, admin, cart, order, notification, and AI routes.
- Sensitive secrets must live in environment variables.
- Role checks must restrict seller/admin operations.

10.3 Reliability

- Order creation should validate cart and product availability before final order creation.
- Payment verification should update payment state only after validation.
- Event publishing should not crash primary user flows if RabbitMQ is unavailable.

10.4 Maintainability

- Each service should own its route, controller, model, and middleware layers.
- Shared behavior should be isolated behind services/helpers rather than duplicated ad hoc.
- AI memory, ranking, and action services should remain independently testable.

11. Tech Stack

Layer	Technology	Purpose
Frontend	React + Vite	Marketplace UI, auth pages, product browsing, cart, checkout, admin and AI components.
Styling	Tailwind CSS	Responsive utility-first UI.
Backend Runtime	Node.js	JavaScript service runtime.
Backend Framework	Express.js	REST routing and middleware.
Database	MongoDB / Mongoose	Users, products, cart, orders, payments, notifications, dashboard data, AI memory.
Cache	Redis	Auth/cache support and service-side acceleration where configured.
Messaging	RabbitMQ	Product, cart, order, payment, notification, and dashboard event flows.
Realtime	Socket.IO	Dashboard/live event features.
Payment	Razorpay	Payment order creation and verification.
AI	Gemini / LangChain + local embeddings	Search intent, chat, semantic ranking, product insight, seller content generation.
Media	ImageKit	Product image upload and delivery.
Testing	Jest	Auth, product, cart, order, and AI integration tests.

12. AI Intelligence Layer

AI Goal: Make marketplace discovery feel like a useful shopping co-pilot instead of a generic chatbot.

- **Intent Understanding** - parses product, category, budget, comparison, mood, and action requests.

- **Semantic Search** - uses local deterministic embeddings to score product meaning even when exact keywords do not match.
- **Personalization** - combines user memory, cart, wishlist, recently viewed products, and order history signals.
- **Persistent Memory** - stores user preference summaries and session history in MongoDB when configured.
- **Action Execution** - can add products to cart, save for later, add to wishlist, or remove from cart through service APIs.
- **Seller AI** - generates product descriptions, bullet points, SEO keywords, categories, subcategories, and tags.
- **Product Insight** - summarizes product fit, risks, similar options, review sentiment, and comparison tradeoffs.

13. Risk Assessment

#	Risk	Likelihood	Impact	Severity	Mitigation
R1	Microservice startup complexity - multiple services must run together for full workflow.	Medium	High	High	Provide scripts/docker compose and clear service dependency docs.
R2	Distributed data consistency - cart/order/payment/product states may drift.	Medium	High	Critical	Use explicit validation, idempotency, lifecycle events, and reconciliation jobs.
R3	AI hallucination - model may invent product details.	Medium	High	High	Restrict prompts to real product data, return product cards, and fallback to deterministic ranking.
R4	RabbitMQ unavailable - event consumers miss dashboard/notification updates.	Medium	Medium	Medium	Keep core flows non-blocking and add retry/dead-letter handling.
R5	Secrets exposure - JWT, payment, email, OAuth, ImageKit, and Gemini keys leak.	Medium	High	High	Use .env, secret rotation, gitignore, and deployment secret stores.
R6	Payment verification edge cases - local test route may be confused with production flow.	Low	High	Medium	Gate test-success route by environment and clearly document production payment path.
R7	AI memory privacy - long-term user preferences must be handled carefully.	Medium	Medium	Medium	Store minimal preference summaries, avoid sensitive data, and support deletion/reset endpoint.

#	Risk	Likelihood	Impact	Severity	Mitigation
R8	Scaling bottleneck - single service instances and local caches limit growth.	Low	Medium	Medium	Introduce gateway, containers, Redis pub/sub, queue workers, and horizontal scaling.

Risk Priority Matrix

	Low Impact	Medium Impact	High Impact
High Likelihood	-	-	-
Medium Likelihood	-	R4, R7	R1, R2, R3, R5
Low Likelihood	-	R8	R6